

บทที่ 6

การเพิ่มประสิทธิภาพการใช้งานเว็บเซอร์วิส

6.1 MSMQ

กล่าวโดยสรุปถึงการทำงานของ MSMQ ว่ามีประโยชน์ต่อแอปพลิเคชันอย่างไรบ้าง

6.1.1 Disconnected Work

ตัวอย่างเช่น เซลล์ขายของ ทำงาน เพิ่มรายชื่อลูกค้าลงไปใน notebook ขณะอยู่บนรถ โดยเมื่อกลับบริษัททันทีที่เชื่อมต่อเข้ากับระบบเน็ตเวิร์ค ต่อกับเซิร์ฟเวอร์ฐานข้อมูลได้ โปรแกรมจะทำการเพิ่มรายชื่อลูกค้าเองให้อัตโนมัติ

6.1.2 Component Available

ปัญหา เซิร์ฟเวอร์ Overload, ปัญหาเน็ตเวิร์ค เราไม่ต้องทำการ โรลแบ็ค (roll back) ทรานแซกชัน ทั้งหมด เนื่องจากงานส่วนนั้นเข้าไปในคิวแล้ว

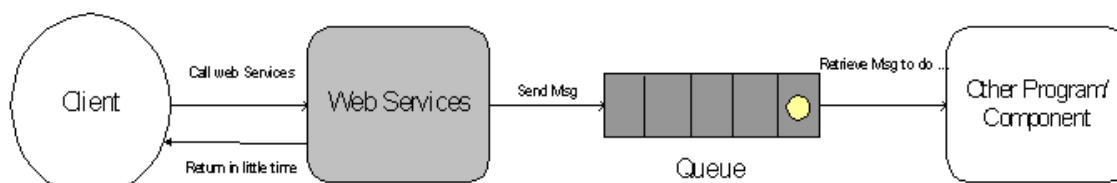
6.1.3 ทรานแซกชัน

- คิวนั้นเป็นส่วนหนึ่งของ ทรานแซกชัน ถ้า ทรานแซกชัน โรลแบ็ค งานในคิวจะถูกลบทิ้งด้วย
- ถ้างานของคิวนั้นเป็น ทรานแซกชัน ถ้าเอาออกมาทำแล้ว ล้มเหลวจะกลับไปเข้าคิวทำใหม่

งานที่เหมาะสมสำหรับคิว

- งานที่สามารถทำได้แน่นอน เช่น การรับ order สั่งสินค้าที่เรารู้ว่า มีจำนวนรองรับเพียงพอแน่นอนเราสามารถที่จะตอบรับลูกค้าได้เลย เพื่อให้ลูกค้าทำงานอย่างอื่นได้ต่อ แล้วทางผู้ขาย(เว็บ)ก็สามารถจะรับคำร้องขอจากลูกค้ารายอื่นต่อได้เลย โดยที่เราทำงานในการจัดเตรียมสินค้านั้น เราสามารถใส่ไว้ในคิวเพื่อทำทีหลังได้
- งานที่ต้องการทำการประมวลผลเป็นระยะเวลานาน เราสามารถรับเรื่องไว้ก่อน โดยใส่ไว้ในคิว เมื่อเราทำงานนั้นๆเสร็จแล้ว ก็อาจจะส่งตอบรับไปยังผู้ใช้ผ่าน อีเมล

6.2 โมเดลการใช้งาน MSMQ



รูปที่ 6.1 MSMQ Model

จากไคลเอนต์ทำการติดต่อไปยังเว็บเซอร์วิส เว็บเซอร์วิสก็จะนำข้อมูลส่งไปยังคิว แล้วทำการตอบรับการทำงานได้ทันทีแบบอสมวาร(Asynchronous) หลังจากนั้นจะเกิดการทริกเกอร์ เรียกใช้คอมโพเนนต์หรือโปรแกรมขึ้นมา ทำการรับข้อมูลในคิวไปใช้ทำงานต่อ เช่น ไปบันทึกลงฐานข้อมูล

6.3 การนำ MSMQ มาเพิ่มประสิทธิภาพเว็บเซอร์วิส

ในการสร้าง เว็บเซอร์วิส แรกของหลายๆคนจะถูกสร้างเป็นแบบ ซิงโครนัส ซึ่งจะประมวลผลบริการที่เข้ามาในขณะที่ทำการร้องขอเข้ามาทันที อย่างไรก็ตาม เว็บเซอร์วิส รูปแบบนี้เกิดข้อจำกัดทาง Scalability สามารถแก้ไขปัญหานี้ได้โดยการใช้ Microsoft Message Queue (MSMQ) กับ เว็บเซอร์วิส ในหัวข้อนี้เราจะทำการยกตัวอย่างระบบที่มีการประมวลผลแบบ อะซิงโครนัส คือ help-desk-request logging

ถ้าสร้างระบบ help-desk-request logging สำหรับ แผนกสารสนเทศขนาดใหญ่ อย่างไรก็ตาม บริษัทมีระบบ request logging อยู่แล้ว แต่ว่า ช้า และล้าสมัย และ ซอฟต์แวร์ ใหม่ ก็จำเป็นต้อง ติดต่อกับระบบเก่านี้ด้วย มันจึงเกิดความซับซ้อน คืออาจจะต้องเขียนคำร้องที่ซ้ำซ้อนไปยังฐานข้อมูลใหม่ด้วย

คุณจะต้องเขียน โค้ดให้ติดต่อกับทั้ง 2 ฐานข้อมูล ดังตัวอย่าง

```

<WebMethod(TRANSACTION_OPTION := _
    TransactionOption.RequiresNew)> _
Public Function SubmitRequest(ByVal request _
    As HelpDeskRequest)
Try
Dim oldWriter as New OldSystemWriter
oldWriter.SubmitRequest(request)

```

```

dim newWriter as New NewSystemWriter
newWriter.SubmitRequest(request)

ContextUtil.SetComplete()

Catch ex As Exception

EventLog.WriteEntry("HelpDeskRequestProcessor", _
ex.Message, EventLogEntryType.Error)

ContextUtil.SetAbort()

End Try

```

โดยจะต้องสร้างจุดในการเชื่อมต่อไปที่ทั้ง 2 ฐานข้อมูลระหว่างการประมวลผล การร้องขอเนื่องจาก ฐานข้อมูลตัวใดตัวหนึ่งหรือสองตัวหยุดทำงานลงหรือ มีการติดต่อไปยัง ฐานข้อมูลพร้อมๆกันมากเกินไป หรือ มีการทำงานอื่นที่ฐานข้อมูลขึ้น ทำให้การตอบสนองของ ซอฟต์แวร์ของเราช้าลงทั้งยังอาจมีปัญหาก็เกิด กับระบบเก่าอยู่แล้ว จึงอาจมีผลกับ request-logging เว็บเซอร์วิส ในหลายทาง ซึ่งเป็นอุปสรรคกับ Scalability ของระบบ

Scalability คือ การวัดความสามารถของ เว็บเซอร์วิส ในการรองรับ จำนวนคำร้องได้อย่างมีประสิทธิภาพโดยไม่เปลี่ยนแปลงสถาปัตยกรรมทางตรรกของ ซอฟต์แวร์ของคุณหรือเปลี่ยนแปลงน้อยที่สุด ซึ่งคุณ จะเห็นปัญหาเรื่อง scalability กับ เว็บเซอร์วิส ที่เป็นแบบ ซิงโครนัส : คำร้องจะถูกยืนยันและประมวลผลและส่ง ผลกลับมายังผู้ใช้ซึ่งเป็นลักษณะการทำงานแบบ เดี่ยว แต่ ละคำร้องจะใช้เวลาในการทำงานและยึดครอง ทรัพยากรบน เว็บเซิร์ฟเวอร์ สำหรับแต่ละคำร้องเป็นเวลานาน ซึ่งการเพิ่มจำนวนคำร้องขึ้นจึงมีแต่ทำให้สถาน การบน เว็บเซิร์ฟเวอร์ แย่ลง

สามารถแก้ปัญหาได้โดยการเปลี่ยนเป็น อะซิงโครนัส เพราะว่า โคลเอนต์ไม่ต้องรอผลตอบสนองของ การ log ทั้งหมดของกระบวนการว่า ผลสำเร็จหรือ ล้มเหลวหรือไม่ แต่ ผู้ใช้จะรอการตอบสนองว่า ได้เริ่มทำ การ log ก็พอแล้วจึงรวดเร็วกว่า และรับประกันผลโดยตัวระบบเองว่าการ log ได้ทำงานอย่างถูกต้อง

โดยประโยชน์นี้ถึงการทำงานจากระบบไปรษณีย์ โดยบุรุษไปรษณีย์ หยิบจดหมายออกจากกล่อง ไปรษณีย์ในช่องส่งที่ละฉบับแล้วทำการส่งไปยังปลายทางจนครบทุกฉบับอาจจะต้องใช้เวลาหลายวันในการ ส่ง ซึ่งเป็นความไม่มีประสิทธิภาพของระบบไปรษณีย์จากตัวอย่าง ความสามารถของ Scalability คือ การที่ บุรุษไปรษณีย์หยิบจดหมายทั้งหมดของคุณรวมทั้งของจดหมายของคนอื่นไปโดยไม่ต้องเปลี่ยนแปลงโครง สร้างของที่ทำกรไปรษณีย์ ซึ่งก็คือกระบวนการส่งได้เสร็จสิ้นแล้วและรับประกันในตัวว่าได้ส่งถึงที่หมายถูก ต้อง

สร้างระบบประมวลผลแบบ อะซิงโครนัส โดยใช้ MSMQ โดย MSMQ ได้มีอยู่ใน Windows 2000 และ Windows XP แล้ว ซึ่งหมายถึงการเก็บข้อความ (message) และทำการส่งหรือรับ ในโครงสร้างที่เรียกว่า

คิว (queue) โดยผ่านการทำงานของ COM+ ทรานแซกชัน ทั้งนี้ MSMQ ยังมีบริการเสริม Windows-based queue security และ การส่งต่อข้อมูลไปยังคิวอื่นแบบอัตโนมัติ

แบ่งงานของการ log ของการร้องขอเป็นสามส่วน เมื่อใช้ MSMQ ในการการสร้าง Scalable งานส่วนแรกคือการส่งคำร้องจากไคลเอนต์ ส่วนที่สองคือการรับคำร้องแล้วใส่เข้าไปในคิวคำร้อง และส่วนที่สามคือนำคำร้องจากคิวแล้วเขียนลงไปที่ฐานข้อมูลทั้งสองคิวจะอยู่ในการทำงานทั้งส่วนที่สองและสาม เว็บเซิร์ฟเวอร์ HTTP รับคำร้องและเพิ่มคำร้องต่างๆเข้าไปในคิวทันที ในขณะที่นั้นองค์ประกอบส่วนอื่นก็จะถือคำร้องต่างๆซึ่งปรากฏในคิว

สร้างคอมโพเนนต์ที่ทำการ trigger เมื่อคิวคำร้องได้รับคำร้องเข้ามา คอมโพเนนต์นี้จะนำข้อความออกจากคิวและทำการเขียนลงฐานข้อมูล โดย เมื่อมี คำร้องเข้าคิวมาจะไปเรียก เมธอดหรือเอกซิกคิว คอมโพเนนต์โดยอัตโนมัติ

```
Public Sub ProcessMessage(ByVal path As String) _
    Implements IMessageHandler.ProcessMessage
    Dim mq As MessageQueue
    Dim m As Message
    Dim oldWriter As IRequestWriter
    Dim newWriter as IRequestWriter
    Try
        mq = New MessageQueue(path)
        m = mq.Receive()
        m.Formatter = New XmlMessageFormatter(_
            New Type() {GetType(_
                VSM_SharedTypes.HelpDeskRequest)})
        oldWriter = New OldSystemRequestWriter()
        oldWriter.WriteRequest(m.Body)
        newWriter = New NewSystemRequestWriter()
        newWriter.WriteRequest(m.Body)
        ContextUtil.SetComplete()
    Catch ex As Exception
        EventLog.WriteEntry("VSM_RequestHandler", _
```

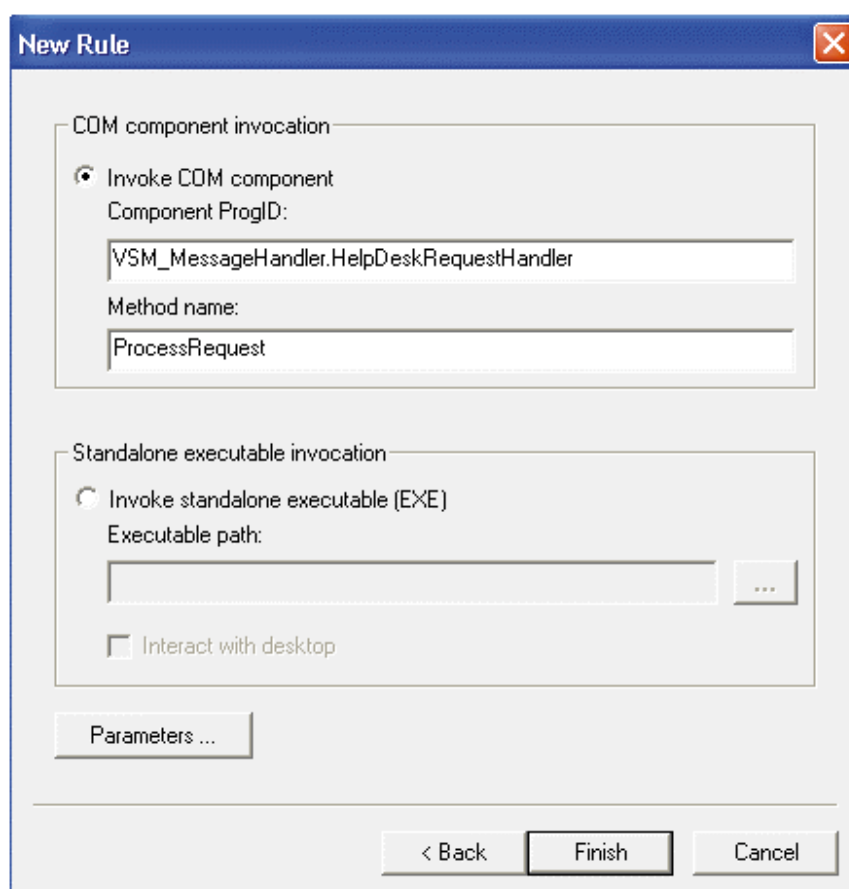
```

ex.Message, EventLogEntryType.Error)
ContextUtil.SetAbort()
Finally
mq.Dispose()
End Try
End Sub

```

จากตัวอย่างข้างบน trigger จะเรียกเมธอดเมื่อมีคำร้องเข้ามาในคิว โดยทำงานอยู่ภายใต้ COM+ ทรานแซกชัน ดึงข้อมูลออกจากคิว และไปบันทึกลงฐานข้อมูลแต่ละตัว จึงเป็นลักษณะงานเดี่ยว (single atomic operation) ถ้าการในการทำงานกับฐานข้อมูลล้มเหลว ก็จะมีการส่งข้อมูลกลับเข้าไปยังคิวใหม่อีกครั้ง หนึ่งด้วย

และกำหนดกฎ (rule) ในการ trigger ของคอมโพเนนต์ ดังรูป



รูปที่ 6-2 การกำหนดกฎ ในการ trigger ของคอมโพเนนต์

หลังจากนี้คุณต้องสร้าง เว็บเซอร์วิส ที่รับคำร้องเข้ามาและใส่เข้าไปในคิว

```
<WebMethod> _
Public Function SubmitOrder(ByVal order As _
HelpDeskRequest)

Dim mqt As New MessageQueueTransaction()
Dim mq As MessageQueue

Try
    mq = New MessageQueue(m_QueuePath)
    mqt.Begin()
    mq.Send(order)
    mqt.Commit()
Catch ex As Exception
    EventLog.WriteEntry("VSM_RequestTaker", _
        ex.Message, EventLogEntryType.Error)
    mqt.Abort()
Finally
    mq.Dispose()
End Try
End Function
```

โดยจากตัวอย่างข้างบนจะทำการรับคำร้องแล้วเขียนลงคิวเท่านั้น โดยมีขนาดเล็ก และเร็ว และในท้ายสุด ก็ต้องทำการสร้าง โค้ดของไคลเอนต์เพื่อนจะยืนยันคำร้องไปยังระบบ ซึ่งผลสุดท้ายเราจะได้ระบบ desk-help-request logging ที่มี scalable และมีความยืดหยุ่น โดย เว็บเซอร์วิส จะรับผิดชอบส่วนการส่งคำร้องไปเข้าคิว ซึ่งทำให้ เว็บเซิร์ฟเวอร์ มีค่ามากที่สุด มีเวลาตอบสนองต่อไคลเอนต์น้อยที่สุด